

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Федеральное государственное автономное образовательное учреждение
высшего образования**

**«Национальный исследовательский технологический университет
«МИСИС»**

Институт компьютерных наук

ОТЧЕТ

по лабораторной работе №2
по дисциплине «Веб-программирование и дизайн»
на тему:
«Разработка прототипа и интеграции»

Выполнил: студент _____

Группа: _____

Проверил: _____

Москва — 2026 г.

Содержание

1 Краткое описание выбранного подхода к реализации

В проекте `alpha_test` реализован прототип корпоративного облачного хранилища. В качестве архитектурного решения выбран модульный подход с разделением на несколько взаимодействующих частей:

- backend на FastAPI (`alpha_test/app`);
- frontend на Flask (`alpha_test/frontend`);
- storage-agent (`alpha_test/agent`) для работы с нодами хранения в polling-режиме.

Таким образом выполнено требование задания о реализации прототипа на базе 2–3 ключевых модулей/сервисов с интеграцией между ними.

2 Структура прототипа

2.1 Перечень модулей/сервисов

- **Auth/Users:** аутентификация, управление учетными записями и ролями.
- **ACL/Resources:** группы доступа, права, ресурсы, связи пользователей и групп.
- **Storage Control:** ноды, тома, выбор места размещения, операции с файлами.
- **Agent Control:** регистрация агентов, heartbeat, очередь заданий, передача файловых данных.
- **Frontend:** пользовательский интерфейс для файловых операций и администрирования.

2.2 Механизмы взаимодействия

- Frontend и backend взаимодействуют через REST API (JSON и multipart).
- Agent взаимодействует с backend через REST API в режиме polling: агент запрашивает задания, выполняет их и возвращает результат.

- Для передачи содержимого файлов используется схема с временными blob-данными и proxy-stream.

3 Описание хранения данных

В качестве СУБД используется SQLite (alpha_test/app.db), что соответствует требованию задания по SQL/NoSQL-хранилищу.

3.1 Список таблиц и полей

Таблица	Ключевые поля
users	id, username, password_hash, role, admin_level, admin_scope_group_id, is_blocked, created_at
groups_acl	id, name, code, group_type, parent_id, created_at
user_groups	user_id, group_id, created_at
resources	id, name, code, path, resource_type, size_limit_mb, parent_id, is_hidden, created_at
resource_folders	id, resource_id, folder_path, created_by, created_at
resource_nodes	resource_id, node_id, created_at
group_permissions	id, group_id, resource_id, permission_code, allow, created_at
nodes	id, name, host, port, connection_type, agent_url, agent_public_key, agent_token, is_active, last_seen, storage_priority, store_all_data
volumes	id, node_id, mount_path, label, quota_gb, quota_bytes, used_gb, used_bytes, is_active, created_at

files	id, file_name, logical_path, size_mb, owner_id, node_id, volume_id, resource_id, storage_rel_path, share_enabled, share_uuid, status, created_at, updated_at
agent_jobs	id, node_id, job_type, payload_json, status, result_json, created_at, updated_at
transfer_blobs	id, file_id, direction, node_id, local_path, size_bytes, status, created_at
audit_logs	id, user_id, username, ip_address, actor_type, event_code, message, meta_json, created_at
service_settings	key, value_json, updated_at

3.2 Соответствие ER-диаграмме из ЛР1

Реализованные таблицы соответствуют основным сущностям, выделенным на этапе ЛР1: пользователи, группы, ресурсы, права, ноды, тома, файлы. Дополнительно добавлены служебные таблицы для интеграции с агентами и аудита всех обращений к системе.

4 Реализация CRUD-функций

4.1 Основные endpoint'ы

Пользователи и аутентификация:

- POST /auth/login;
- POST /auth/users, GET /auth/users, PUT /auth/users/{id}, DELETE /auth/users/{id}.

ACL и ресурсы:

- POST /acl/groups, GET /acl/groups, DELETE /acl/groups/{id};
- POST /acl/resources, PUT /acl/resources/{id}, DELETE /acl/resources/{id};
- POST /acl/permissions/grant, DELETE /acl/permissions/{id}.

Ноды, тома, файлы:

- POST /nodes, PUT /nodes/{id}, DELETE /nodes/{id};
- POST /volumes, PUT /volumes/{id}, DELETE /volumes/{id};
- POST /files/upload, GET /files, PUT /files/{id}, DELETE /files/{id}.

4.2 Пример тестового вызова

```
POST /auth/login
{
  "username": "admin",
  "password": "admin123"
}
```

В ответе возвращается JWT-токен, используемый для вызова защищенных endpoint'ов.

5 Реализация безопасности

- Используется JWT-аутентификация для защищенных API.
- Применяется авторизация по ролям и ACL-пермишенам: view/read/write/delete/share/manage_* / admin_panel.
- Для неавторизованных запросов к закрытым операциям возвращается 401 Unauthorized.
- Для недостатка прав возвращается 403 Forbidden.
- Реализованы защищенные операции скачивания, включая приватные ссылки формата /private/{uuid}/download.

6 Применение принципов SOLID, DRY и KISS

SOLID:

- принцип SRP соблюден за счет разделения слоя роутеров, сервисов и репозиториев;
- бизнес-логика не дублируется в контроллерах API.

DRY:

- общие проверки доступа вынесены в AccessService;
- повторяемые SQL-операции инкапсулированы в репозиториях;
- аудит HTTP-запросов реализован централизованно через middleware.

KISS:

- архитектура построена на понятном стеке FastAPI + Flask + SQLite;
- интеграция выполнена через REST без избыточной инфраструктуры.

7 Выводы

В результате выполнения лабораторной работы №2 реализован работоспособный прототип, который:

- содержит несколько интегрированных модулей/сервисов;
- обеспечивает хранение данных и CRUD-операции;
- реализует аутентификацию и авторизацию доступа;
- демонстрирует применение принципов SOLID, DRY и KISS;
- может быть расширен до полнофункциональной производственной системы.